



Séptima Unidad

SQL

**Lenguaje de
consultas
Relacionales**

Introducción

Los lenguajes formales proporcionan una **notación concisa** para la representación de consultas.

Sin embargo, los sistemas de base de datos necesitan un **lenguaje de consultas más cómodo** para el usuario.

Aunque SQL se considere un lenguaje de consultas, contiene **muchas otras capacidades** que incluyen características para definir estructuras de datos, modificación de datos y la **especificación de restricciones de integridad**.

Los sistemas de gestión de base de datos con soporte SQL más utilizados son:
DB2, Oracle, SQL Server, Sybase ASE, MySQL, PostgreSQL, Firebird

Historia

SQL se ha establecido como el lenguaje estándar de base de datos relacionales. Hay **numerosas versiones** de SQL.

La versión original se desarrolló en el laboratorio de investigación de San Jose, California (**San Jose Research Center**) de IBM, este lenguaje originalmente denominado **Sequel**, se implementó como parte del proyecto System R, a principios de 1970 [McJones97].

Desde entonces ha evolucionado a lo que ahora se conoce como **SQL (Structured Query Language)**, lenguaje estructurado de consultas).

Evolución

En 1986, **ANSI** (American National Standards Institute, Instituto Nacional Americano de Normalización) e ISO (International Standards Organization, Organización Internacional de Normalización) publicaron una norma de SQL denominada **SQL-86**.

En 1987 IBM publicó su propia norma de SQL denominada **SAA-SQL** (System Application Architecture Database Interfaz, *Interfaz de bDs para arquitecturas de aplicación de sistemas*).

Años más tarde SQL3 incorpora **consultas recursivas** y **Triggers** y luego el SQL2003 incorpora **XML**.

SQL proporciona más de dos tipos de lenguajes diferentes: en particular veremos comandos de **DDL**, para **especificar el esquema relacional** y de **DML** para **las consultas y actualizaciones** de la base de datos.

SQL no es un lenguaje de programación

Fue diseñado con el único propósito de **acceder a datos estructurales**

Como en el caso de los más modernos lenguajes relacionales, SQL está basado en el **cálculo relacional de tuplas**.

Toda consulta formulada utilizando el **cálculo relacional** de tuplas (o su equivalente, el **álgebra relacional**) se puede formular también utilizando SQL.

Características

El SQL es un lenguaje de acceso a bases de datos que explota la flexibilidad y potencia de los sistemas relacionales permitiendo gran variedad de operaciones s/los mismos.

Es un lenguaje **declarativo de alto nivel**, que gracias a su **fuerte base teórica** y su **orientación al manejo de conjuntos** de registros, y no a registros individuales, permite una alta productividad en codificación

De esta forma **una sola sentencia puede equivaler a uno o más programas** que utilizan un leng. de bajo nivel orientado a registro.

Otras características de SQL

SQL puede ser utilizado para otras características propias que no poseen los lenguajes formales de consultas:

Características

Definición de vistas.

El DDL de SQL incluye instrucciones para la definición de vistas.

Autorización.

El DDL de SQL incluye instrucciones para la especificación de los derechos de acceso a los objetos de la BD.

Integridad.

El DDL de SQL también incluye un conjunto de sentencias para la especificaciones de restricciones de integridad.

Control de transacciones. SQL incluye ordenes para la especificación de los estados de una transacción, algunas implementaciones permiten además el bloqueo explícito de objetos de datos con el fin de manejar control de concurrencia.

Otras características de SQL (2)

Capacidades que van más allá del cálculo o del álgebra relacional. Características EXTRAS proporcionadas por SQL:

Capacidades aritméticas

- En SQL es posible incluir operaciones aritméticas así como comparaciones, por ejemplo $A < B + 3$.

Asignación y comandos de impresión

- Es posible imprimir una relación construida por una consulta y asignar una relación calculada a un nombre de relación.

Funciones agregadas

- Operaciones tales como *promedio* (*average*), *suma* (*sum*), *máximo* (*max*), etc., se pueden aplicar a las columnas de una relación para obtener una cantidad única.

Funcionalidad



SQL

- El SQL proporciona funcionalidad *más allá de la simple consulta* (recuperación) de datos.



Asume el papel de:

- lenguaje de definición de datos (**LDD**),
- lenguaje de definición de vistas (**LDV**)
- lenguaje de manipulación de datos (**LMD**).



Además permite:

- la concesión y denegación de **permisos**,
- la implementación de **restricciones de integridad** y controles de **transacción**,
- y la **alteración de esquemas** (estructura).

Lenguaje de Definición de datos (DDL)

DDL

- Un esquema de bases de datos se representa mediante un sub_lenguaje especial llamado lenguaje de definición de datos

Componente estática

- El resultado de la compilación de estas instrucciones es un conjunto de tablas, relaciones y reglas cuyas definiciones quedan almacenadas en un archivo

Metadatos

- Este archivo comúnmente llamado diccionario de datos (o catalogo del sistema) es el que se consulta toda vez que se quiere leer, modificar o eliminar los datos de la bd

Lenguaje de Manipulación de datos (DML)

Se entenderá por manipulación de datos la:



Recuperación de Información. **CONSULTAS**



Inserción de nueva Información.



Eliminación de información existente.



Modificación de Información Almacenada.

Sentencia SELECT

SELECT [ALL | DISTINCT]

{ * | *expr_1* [AS *c_alias_1*] [, ... [, *expr_k* [AS
c_alias_k]]]}

FROM *table_name_1* [*t_alias_1*] [, ... [,
table_name_n [*t_alias_n*]]]

[**WHERE** *condition*]

[**GROUP BY** *name_of_attr_i* [,... [, *name_of_attr_j*]]

[**HAVING** *condition*]]

[{**UNION** [ALL] | **INTERSECT** | **EXCEPT**} SELECT ...]

[**ORDER BY** *name_of_attr_i* [ASC | DESC] [, ... [,
name_of_attr_j [ASC | DESC]]]];

Cláusulas comando SELECT



- Indica que se incluyen **TODOS** los atributos, a menos que se mencione una lista

FROM

- Indica de cuál/es tabla/s se extraerá la información

WHERE

- Permite el **filtrado de filas**, es decir, explicitar una condición de búsqueda de tuplas

ORDER BY

- Permite indicar el o los campos que liderarán y mostrarán el ordenamiento de las tuplas.

Algunos Ejemplos

Todos los atributos
de la tabla

Select * from Clientes

Select * from Ventas

Especificación de atributos → **Proyección**

Select apellido, nombres, teléfono from Clientes

Select monto*2 as **doble**, fecha from Ventas

Nombre con el que
aparece la
columna del
campo calculado

Select * from Clientes **order by** provincia

Select * from Alumnos **order by** carrera

Select * from Alumnos **order by** apellido

Select * from Alumnos **group by** carrera

Se agrupan por un
campo común

Cláusulas comando SELECT (2)

GROUP BY

- Permite “agrupar” tuplas **según un campo común**. Permite funciones de agregación

HAVING

- Permite un nuevo filtrado, **pero sobre las tuplas afectadas por el GROUP BY**, en función de una condición aplicable a cada grupo de filas

UNION [ALL] | INTERSECT | EXCEPT

- permiten implementar **otras operaciones** del AR, y en especial, para la **implementación de subconsultas o consultas anidadas**

Condiciones WHERE

- **Comparación: Operadores >, <, >=, <= y <>.**

```
SELECT campos FROM tabla WHERE campo > valor
```

- **Pertenencia a rango: Operador BETWEEN:**

```
SELECT campos FROM tabla  
WHERE campo BETWEEN inferior AND superior
```

- **Pertenencia a conjunto: Operador IN** seguido de una lista de elementos entre paréntesis.

```
SELECT campos FROM tabla  
WHERE campo IN (valor, valor, valor, ...)
```


Condiciones WHERE (2)

- **Correspondencia con patrón: Operador LIKE**

```
SELECT campos FROM tabla  
WHERE campo LIKE patron
```

El patrón permite búsquedas con **comodines**:

“%” Representa 0 ó más caracteres

“?” Representa un carácter

- **Test de valor nulo: El operador IS NULL**
permite determinar si existe ó no valor:

```
SELECT campos FROM tabla  
WHERE campo IS NULL
```

Más Ejemplos (like)

*Mostrar todos los empleados
de nombre Angel*

```
select Apenom from Empleados  
where Apenom LIKE 'Angel%'
```

```
select Apenom from Empleados  
where Apenom LIKE '%ANGEL%'
```

Sólo encuentra los
que tienen "Angel"
como **PRIMER
NOMBRE**

Encuentra los
empleados que
tienen "Angel" como
**PRIMER o SEGUNDO
NOMBRE**

Más Ejemplos (between)

Incluye los
que ganan
\$30000 y
\$40000,

- *Listar a los empleados que ganan entre \$30000 y \$40000*

```
select Empleado, sueldo from Empleados  
where SUELDO >=30000 and sueldo<=40000
```

EQUIVALENTE

```
select Empleado, sueldo  
from Empleados  
where SUELDO between 30000 and 40000
```

EQUIVALENTE

Más Ejemplos (IN)

- *Listar hora que se dictan clases los lunes, miércoles y viernes*

Las clases que se dictan los lunes o los
miércoles o los viernes

```
select dia, hora from Clases  
where dia="Lunes" or dia="Miercoles" or dia="Viernes"
```

EQUIVALENTE

```
select dia, hora  
from Clases  
where dia IN (Lunes, Miércoles, Viernes)
```

EQUIVALENTE



Condiciones WHERE ₍₃₎



En una cláusula WHERE Se pueden combinar **múltiples condiciones** (de cualquiera de los tipos) mediante:

Los operadores lógicos **AND, OR y NOT**, determinando con el uso de **paréntesis** el orden de evaluación

(y contrarrestando el efecto de la prioridad de operadores).

Comando SELECT (otros ejemplos)

```
SELECT * FROM EMPLEADO  
WHERE SALARIO > 35000
```

```
SELECT APELLIDO, SEXO, SALARIO FROM  
EMPLEADO WHERE SALARIO > 35000;
```

```
SELECT APELLIDO, SEXO, SALARIO FROM EMPLEADO  
WHERE SEXO = 'F' AND  
(APELLIDO="PEREZ"  
OR APELLIDO="DIAZ");
```

Comando SELECT (Ejemplos)

Queremos conocer cuánto es el descuento para la jubilación del empleado (aporte jubilatorio), consistente en el 13% del salario, y listar sólo de los que ése monto no supere los \$25000:

```
SELECT APELLIDO, SALARIO*0.13 AS aporte_jub  
FROM EMPLEADO  
WHERE SALARIO*0.13 <= 25000
```

Cómo ver datos de dos o más tablas

Para poder incluir datos contenidos en atributos dispuestos en más de una tabla, es necesario que éstas tablas se encuentren vinculadas o “**relacionadas**” a partir de atributos o **campos comunes**.

```
SELECT APELLIDO, SALARIO, CARGO  
FROM EMPLEADO, CARGOS  
WHERE  
EMPLEADO.ID_CARGO=CARGO.ID_CARGO
```

Si la cláusula “where” se omite ¿qué sucede?

Comando SELECT (**sin** Integridad Referencial)

Es posible **listar campos de tablas relacionadas entre sí**, utilizando CAMPOS COMUNES mediante cláusula "where" **SIN** utilizar *integridad referencial* (JOIN).

En la cláusula FROM ES POSIBLE INTRODUCIR un **alias** al nombre para cada relación o tabla, en especial en casos donde existan atributos con nombre común en las tablas.

Luego podemos distinguir entre los atributos con nombre común simplificando con la **adición de un prefijo al nombre del atributo con el nombre del alias seguido de un punto**.



Comando SELECT (sin Integridad Referencial)



```
SELECT L.titulo, L.editorial, AU.apellido  
FROM LIBRO L, AUTORES AU  
WHERE L.id_autor = AU.id_autor
```



Comando SELECT (sin Integridad Referencial) ₂

```
SELECT E.apellido, P.nombrep, T.horas  
FROM EMPLEADO E, PROYECTO P, TRABAJAEN T  
WHERE T.nsse = E.nss  
AND  
T.nump = P.numerop;
```

Comando SELECT (con Integridad Referencial)

```
SELECT L.titulo, L.editorial, AU.apellido  
FROM LIBRO L INNER JOIN AUTORES AU  
ON L.id_autor = AU.id_autor
```

Si los campos se tienen IGUAL nombre, en lugar de ON se utiliza "USING"

```
SELECT L.titulo, L.editorial, AU.apellido  
FROM LIBRO L INNER JOIN AUTORES AU  
USING (id_autor)
```



Comando SELECT (con Integridad Referencial)

```
SELECT E.apellido, P.nombrep, T.horas  
FROM EMPLEADO E INNER JOIN TRABAJAEN T  
ON (T.nsse = E.nss)  
INNER JOIN PROYECTO P  
ON (T.nump = P.numerop)
```

EL JOIN EN EL LENGUAJE RELACIONAL

La sentencia JOIN del SQL permite combinar registros de una o más tablas en una base de datos relacional.

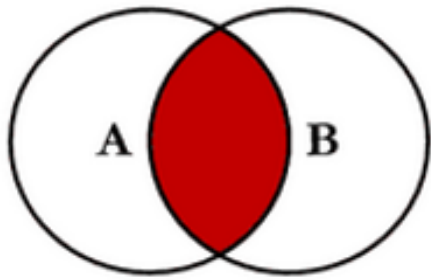
En el Lenguaje de Consultas Estructurado (SQL) y el estándar ANSI se especifican cuatro tipos de JOIN:

INNER

LEFT

RIGHT

FULL OUTER



INNER JOIN

Con esta operación cada registro en la tabla A es combinado con los correspondientes de la tabla B que satisfagan las condiciones que se especifiquen en el predicado del *JOIN*.

Cualquier registro de la tabla A o de la tabla B que no tenga uno correspondiente en la otra tabla es **excluido**, y solo aparecerán los que tengan correspondencia en la otra tabla.

Este es el tipo de *JOIN* más utilizado, por lo que es considerado el **tipo de combinación predeterminado**.



```

1 • select c.apellido,c.nombre,c.id_cliente,p.id_cliente,p.fech_pedido,p.id_pedido
2   from pedidos p,clientes c; PRODUCTO CARTESIANO
3
4 • Select c.apellido,c.nombre,c.id_cliente,p.id_cliente,p.fech_pedido,p.id_pedido
5   from pedidos p,clientes c
6   where p.id_cliente=c.id_cliente; PRODUCTO CARTESIANO CON
7                                     IGUALDAD EN CAMPO COMÚN
8 • Select c.apellido,c.nombre,c.id_cliente,p.id_cliente,p.fech_pedido,p.id_pedido
9   from pedidos p inner join clientes c using (id_cliente); INNER JOIN
10

```



Result Grid Filter Rows: Export: Wrap Cell Content:

	apellido	nombre	id_cliente	id_cliente	fecha_pedido	id_pedido
▶	Tienza	Carmen	1	1	1995-11-02 00:00:00	6
	Tienza	Carmen	1	1	1995-12-14 00:00:00	300
	Guardiola Bartolome	Jose Vicente	2	2	1995-12-02 00:00:00	214
	Guardiola Bartolome	Jose Vicente	2	2	1995-07-23 00:00:00	363
	Alcolea Alemani	Sergio	3	3	1995-02-20 00:00:00	345
	Artero Salcedo	Jesus	4	4	1995-11-21 00:00:00	141

Result 6 x

Output

Action Output

	#	Time	Action	Message
✓	85	16:25:36	select c.apellido,c.nombre,c.id_cliente,p.id_cliente,p.fech_pedido,p.id_pedido from pedid...	132275 row(s) returned
✓	86	16:25:46	Select c.apellido,c.nombre,c.id_cliente,p.id_cliente,p.fech_pedido,p.id_pedido from pedi...	407 row(s) returned
✓	87	16:30:49	Select c.apellido,c.nombre,c.id_cliente,p.id_cliente,p.fech_pedido,p.id_pedido from pedi...	407 row(s) returned



ANEXO

MySQL

y

Workbench

Introducción

Inicialmente, MySQL carecía de elementos considerados esenciales en las bases de datos relacionales, tales como integridad referencial y transacciones.

A pesar de ello, atrajo a los desarrolladores de páginas web con contenido dinámico, justamente por su simplicidad

MySQL es un motor de bases de datos de muy rápida en la lectura, pero puede provocar problemas de integridad en entornos de alta concurrencia en la modificación. **En aplicaciones web hay baja concurrencia en la modificación de datos y en cambio el entorno es intensivo en lectura de datos, lo que hace a MySQL ideal para este tipo de aplicaciones.**

¿Qué es MySQL?

MySQL

es un dbms (sistema de gestión de base de datos) relacional desarrollado

bajo licencia

dual GPL/Licencia

comercial por Oracle Corporation y está considerada como la base datos open source más popular del mundo, y una de las más populares en general junto a Oracle y Microsoft SQL Server, sobre todo para entornos de **desarrollo web**

MySQL

MySQL fue inicialmente desarrollado por MySQL AB (empresa fundada por David Axmark, Allan Larsson y Michael Widenius).

MySQL A.B. fue adquirida por Sun Microsystems en 2008, y ésta a su vez fue comprada por Oracle Corporation en 2010, la cual ya era dueña desde 2005 de Innobase Oy empresa finlandesa desarrolladora del motor InnoDB para MySQL.

Evolución

- Poco a poco los elementos de los que carecía MySQL han ido siendo incorporados tanto por desarrollos internos, como por desarrolladores de software libre. Características disponibles en las últimas versiones:

Amplio subconjunto del lenguaje SQL.

Disponibilidad en gran cantidad de plataformas y sistemas.

Posibilidad de selección de mecanismos de almacenamiento que ofrecen diferentes velocidades de operación, soporte físico, capacidad, distribución geográfica, transacciones...

Transacciones y claves foráneas.

Conectividad segura. Replicación.

Búsqueda e indexación de campos de texto.

Características

Multiplataforma

- Para S.O. como Windows, Linux y Mac disponemos de nuestro servidor para instalarlo.

Multihilos

- Está optimizado para equipos de múltiples procesadores

Fácil encontrar ayuda

- Al ser una base de datos que se utiliza en multitud de aplicaciones web existen multitud de tutoriales, foros, en la red en los que podemos encontrar la información que necesitamos

Fácil de aprender

- Simplemente con conocer el estándar de SQL podemos manejar la base de datos MySQL sin ningún problema

Características

Probado

- MySQL es un DBMS **ampliamente probado** por distintos usuarios y empresas con alto éxito.

Menos mantenimientos

- Realmente esto nos da la ventaja para que un programador cualquiera pueda aprender rápidamente como debe mantener la base de datos para sus aplicaciones. **Sin necesidad de ser un experto Administrador en Base de Datos (DBA).**

otras

- Bases de datos como Oracle requieren de DBA para la gestión de su información debido a todas las características que tienen para su administración. En cambio MySQL para el funcionamiento habitual de una aplicación incluye unas características mínimas que nos sirven ampliamente para nuestras aplicaciones sin tener que recurrir aun DBA para que administre la base

Código Abierto o no?

A diferencia de proyectos como Apache, donde el software es desarrollado por una comunidad pública y los derechos de autor del código están en poder del autor individual,

MySQL es patrocinado por una empresa privada, que posee el copyright de la mayor parte del código

una *Community*, distribuida bajo la Licencia pública general de GNU, versión 2, y

varias versiones *Enterprise*, para aquellas empresas que quieran incorporarlo en productos privativos.

Esto es lo que posibilita el **esquema de doble licenciamiento**: la base de datos se distribuye en varias versiones

Las versiones *Enterprise* incluyen productos o servicios adicionales tales como herramientas de monitorización y soporte oficial.

Herramientas

Cada distribución de MySQL incluye herramientas para trabajar directamente con las bases de datos, especialmente las interfaces gráficas como Workbench

MySQL Workbench es una **herramienta visual de diseño de bases de datos** que integra **desarrollo de software, Administración de bases de datos, diseño de bases de datos, creación y mantenimiento** para el administrador de sistemas de base de datos MySQL.

MySQL Workbench es uno de los primeros productos de la familia MySQL que ofrece dos ediciones diferentes - una open source y una edición comercial



¡¡Muchas Gracias !!

Consultas en el foro

